Forms: Controlled vs Uncontrolled Components

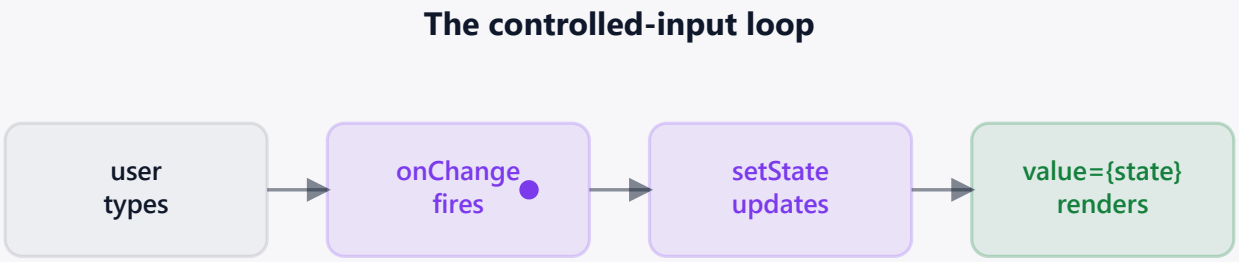
6 min read Updated 26 Mar 2026 Topic 7 of 62

1.7 — Forms: Controlled vs Uncontrolled Components

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is a controlled component?"	🟢	★★★★★
"What is an uncontrolled component?"	🟢	★★★★☆
"When would you use one over the other?"	🟡	★★★★☆
"How do you handle multiple inputs in one form?"	🟢🟡	★★★★★
"How do you handle form validation in React?"	🟡	★★★★☆

Controlled Components



React state is the single source of truth: every keystroke round-trips through state and back into the input.

A controlled component is a form element whose value is driven by React state. React owns the data. The DOM just displays it.

The pattern is always the same: the `value` prop sets what the input shows, and the `onChange` handler updates the state whenever the user types.

JSX Copy

```jsx
function ControlledInput() {
  const [name, setName] = useState('');

  return (
    <div>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
      <p>You typed: {name}</p>
    </div>
  );
}
```

The reason this is called "controlled" is because React controls the value at all times. If you remove the `onChange` handler, the input becomes read-only because React forces the value to stay the same.

Handling Multiple Inputs with One Handler

Writing a separate handler for every field gets old fast. Use the input's `name` attribute and computed property names instead.

JSX

Copy

```jsx
function SignupForm() {
  const [formData, setFormData] = useState({
    username: '',
    email: '',
    password: '',
  });

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value,
    }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="username" value={formData.username} onChange={handleCha
      <input name="email" type="email" value={formData.email} onChange={ha
      <input name="password" type="password" value={formData.password} onC
      <button type="submit">Sign Up</button>
    </form>
  );
}
```

This single `handleChange` works for text, email, password, and checkbox inputs. The `[name]` syntax dynamically sets the right key in the state object.

Uncontrolled Components

An uncontrolled component lets the DOM handle the form data. You access the value when you need it (usually on submit) using a `ref`.

JSX Copy

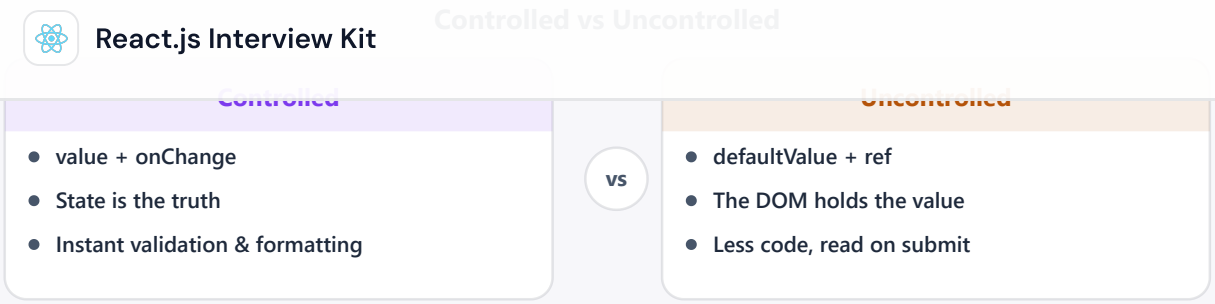
```jsx
function UncontrolledForm() {
  const nameRef = useRef();
  const emailRef = useRef();

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Name:', nameRef.current.value);
    console.log('Email:', emailRef.current.value);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input ref={nameRef} placeholder="Name" />
      <input ref={emailRef} type="email" placeholder="Email" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

There's no `value` prop, no `onChange`, no state. The browser manages the input, and you read the value from the DOM when you need it.

When to Use Which

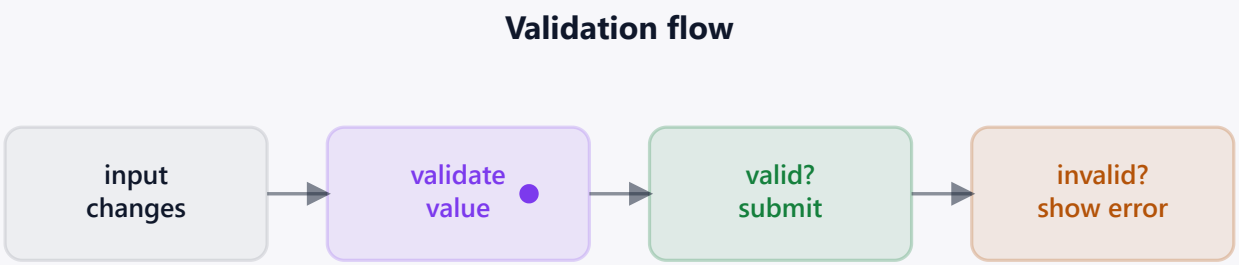


Default to controlled; reach for uncontrolled for simple forms or file inputs.

Situation	Controlled	Uncontrolled
Real-time validation (show errors as user types)	✅	❌
Conditional disabling of submit button	✅	❌
Enforcing input format (uppercase, phone mask)	✅	❌
Multiple inputs that depend on each other	✅	❌
Simple form, just need value on submit	Either works	✅ simpler
Integrating with non-React code or third-party libs	❌ harder	✅
File inputs (`<input type="file">`)	❌ always uncontrolled	✅

Default recommendation: Use controlled components. They give you full power over the data at all times. Fall back to uncontrolled only for simple cases or file inputs.

Form Validation



Validate on change (or on submit), then branch: enable submit when valid, surface a message when not.

Interviewers want to see that you can validate on submit AND provide inline feedback as the user types.

JSX Copy

```jsx
function ValidatedForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState({});
  const [touched, setTouched] = useState({});

  const validate = () => {
    const newErrors = {};
    if (!email.includes('@')) newErrors.email = 'Must be a valid email';
    if (password.length < 8) newErrors.password = 'Must be at least 8 char
    return newErrors;
  };

  const handleBlur = (field) => {
    setTouched(prev => ({ ...prev, [field]: true }));
    setErrors(validate());
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const validationErrors = validate();
    setErrors(validationErrors);
    setTouched({ email: true, password: true });
```

```
  };

  return (
    <form onSubmit={handleSubmit}>
      <div>
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          onBlur={() => handleBlur('email')}
          placeholder="Email"
        />
        {touched.email && errors.email && <span className="error">{errors.
      </div>
      <div>
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          onBlur={() => handleBlur('password')}
          placeholder="Password"
        />
        {touched.password && errors.password && <span className="error">{e
      </div>
      <button type="submit">Login</button>
    </form>
  );
}
```

The `touched` state ensures errors only show after the user has interacted with a field, not immediately on page load.

Common Mistakes

Mistake 1: Forgetting `e.preventDefault()` on form submit. Without it, the browser reloads the page.

Mistake 3: File inputs cannot be controlled. `<input type="file">` is always uncontrolled because its value is read-only for security reasons. Use a ref or the `onChange` event to access the selected file.

🔥 Output Question

JSX Copy

```jsx
function App() {
  const [value, setValue] = useState('hello');

  return <input value={value} />;
}
```

What happens? The input shows "hello" and the user cannot type anything. There's no `onChange` handler, so React keeps forcing the value back to "hello" on every render. This is a read-only controlled input. You'll also see a React warning in the console about providing `value` without `onChange`.

🗣️ Interview Answer

> "A controlled component is a form element where React state is the single source of truth for its value. You bind the value to state and update it through an onChange handler. An uncontrolled component lets the DOM manage the value, and you access it with a ref when needed. I default to controlled components because they give you real-time access to the data for validation, conditional logic, and formatting. Uncontrolled is useful for simple forms or file inputs."

- **Controlled:** React state drives the input via `value` + `onChange`. Source of truth is React.
- **Uncontrolled:** DOM drives the input. Access value via `ref`. Source of truth is the DOM.
- Use **computed property names** (`[e.target.name]`) for a single handler across multiple inputs.
- **File inputs** are always uncontrolled.
- Validate on **blur** for inline feedback, on **submit** for final check.
- Track **touched** state to avoid showing errors before the user has interacted.

QUICK CHECK — no marks, just for you.

1. A component renders an input whose value prop is bound to a state variable holding the string "hello", but no onChange handler is passed. What does the user experience when they try to type into it?

- ○ The input shows "hello" and typing works, but React throws an error on the first keystroke
- ○ The input renders empty, because React ignores a value prop that has no onChange to pair with
- ○ The input shows "hello" and typing does nothing, because React keeps forcing the value back on every render
- ○ The input shows "hello" and typing works normally, but the state variable is never updated

Check answer

2. A signup form keeps username, email, and password in one state object and wants a single handleChange for all three inputs. What makes one shared handler able to update the correct key?

Each input needs its own key prop, which React passes to the handler so it knows which field changed

- Each input's name attribute is read off the event target and used as a computed property key when spreading into new state
- The handler must inspect e.target.value and infer the field from the value's shape or format
- React automatically matches an input to the state key with the same type attribute

Check answer

3. In the validated login form, errors are computed by validate() but rendered only when both touched[field] and errors[field] are truthy. What does the touched state specifically buy you?

- It stops validate() from running more than once per field, avoiding redundant work on every keystroke
- It prevents validation errors from appearing before the user has interacted with that field, such as on first page load
- It marks fields as required so that submit is blocked until each one has been filled in
- It caches the last error message so the field keeps showing it even after the value becomes valid

Check answer

React.js Interview Kit

ARTICLES & DOCS

- Reacting to Input with State
  react.dev
- <input> — controlled inputs
  react.dev
- Sharing State Between Components
  react.dev

VIDEOS



Controlled vs uncontrolled — explained clearly
YouTube



Master React forms: controlled vs uncontrolled
YouTube



Controlled vs uncontrolled — interview take
YouTube

Hand-picked references for this lesson.